Blockme

No-Turn-Back Social Media Blocker

Implementation Plan — Full Specification

VPN-Based | Cross-Device | Tamper-Proof | Typed Friction | Emergency Cap

Generated April 2026

Blockme — No-Turn-Back Social Media Blocker

Implementation Plan

Context

Build a self-hosted, tamper-proof social media blocker that works on every device (iPhone, Android, tablet, laptop) on any network (home WiFi, 4G/5G, hotel WiFi), blocks both native apps and browsers, and makes bypassing genuinely painful. Informed by gaps in Cold Turkey, AppBlock, Brick, and Opal.

Architecture

```
User Device (any OS, any network)
      │  WireGuard VPN tunnel — full tunnel (0.0.0.0/0)
      ▼
Blockme Server (Linux VPS or home server)
      ├── WireGuard VPN  :51820 UDP
      ├── DNS blocker    :53    (VPN interface only)
      │     blocked → 0.0.0.0 / ::
      │     allowed → upstream 8.8.8.8
      ├── Express API    :3000
      └── React dashboard
```

Why WireGuard: Router-DNS only works on home WiFi. WireGuard routes ALL traffic (including native app traffic) through the server regardless of network. DNS in client config points to Blockme server → no DNS leak possible.

Tech Stack

Layer	Choice
Backend	Node.js + Express + TypeScript
VPN	WireGuard (system) via `wg` / `wg-quick` shell calls
DNS	`dns2` npm package
Database	SQLite + `better-sqlite3`
Frontend	React + TypeScript + Vite + Tailwind CSS v4
Auth	`bcrypt` + `jsonwebtoken`
Scheduling	`node-cron`
QR codes	`qrcode` npm
Rate limiting	`express-rate-limit`
NTP check	`ntp-client` npm (time-change detection)

Directory Structure

```
/home/user/Blockme/
├── package.json                    # npm workspaces root
├── .gitignore
├── README.md
├── blockme.service                 # systemd unit file
│
├── server/
│   ├── package.json
│   ├── tsconfig.json
│   └── src/
│       ├── index.ts                # Express + startup + health loop
│       ├── dns-server.ts           # DNS blocking engine
│       ├── wireguard.ts            # WireGuard peer management
│       ├── blocklist.ts            # 200+ domain list + matching
│       ├── scheduler.ts            # Schedule checker (NTP-based time)
│       ├── session-manager.ts      # Session lock state machine ★
│       ├── friction.ts             # Typed friction phrases + escalation ★
│       ├── emergency.ts            # Emergency unlock system ★
│       ├── health-monitor.ts       # Reliability monitoring loop ★
│       ├── time-guard.ts           # Clock manipulation detection ★
│       ├── db/
│       │   ├── index.ts
│       │   ├── schema.ts           # All tables + seed data
│       │   └── queries.ts          # All SQL functions (typed)
│       └── routes/
│           ├── auth.ts
│           ├── session.ts          # Session lock + unlock flow ★
│           ├── devices.ts          # VPN peer management + QR
│           ├── blocklist.ts
│           ├── schedules.ts
│           ├── stats.ts
│           └── health.ts           # Health status endpoint ★
│
├── client/
│   ├── package.json
│   ├── tsconfig.json
│   ├── vite.config.ts
│   ├── postcss.config.ts
│   ├── index.html
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── api/client.ts
│       ├── contexts/AuthContext.tsx
│       └── pages/
│           ├── LoginPage.tsx
│           ├── DashboardPage.tsx   # Status + health indicator + stats
│           ├── DevicesPage.tsx     # QR code device setup
│           ├── BlocklistPage.tsx   # Per-platform + URL rule toggles
│           ├── SchedulesPage.tsx   # Locked once active
│           ├── UnlockPage.tsx      # Typed friction unlock flow ★
│           ├── EmergencyPage.tsx   # Emergency-only access ★
│           └── SettingsPage.tsx
│
└── extension/                      # Chrome MV3 / Firefox
    ├── manifest.json
    ├── background.ts               # URL-pattern blocking (e.g. YT Shorts)
    └── popup/
        ├── popup.html
        └── popup.ts
```

Database Schema

```
-- Admin auth
CREATE TABLE admin (
  id            INTEGER PRIMARY KEY,
  password_hash TEXT NOT NULL
);

-- Key/value settings store
-- Keys: blocking_enabled, upstream_dns, protection_enabled,
--       server_ip, wg_server_private_key, wg_server_public_key,
--       emergency_cap_weekly (default '3'),
--       ntp_server (default 'pool.ntp.org'),
--       health_status ('active'|'degraded'|'broken'),
--       last_health_check (ISO timestamp)
CREATE TABLE settings (key TEXT PRIMARY KEY, value TEXT NOT NULL);

-- Platform registry (pre-seeded)
CREATE TABLE platforms (
  id       INTEGER PRIMARY KEY AUTOINCREMENT,
  name     TEXT NOT NULL,
  category TEXT NOT NULL,   -- 'social','video','messaging','professional'
  enabled  INTEGER NOT NULL DEFAULT 1
);

-- DNS domain rules
CREATE TABLE domains (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  platform_id INTEGER REFERENCES platforms(id),
  domain      TEXT NOT NULL UNIQUE,
  is_custom   INTEGER NOT NULL DEFAULT 0,
  enabled     INTEGER NOT NULL DEFAULT 1
);

-- Sub-platform URL pattern rules (for browser extension)
CREATE TABLE url_rules (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  platform_id INTEGER REFERENCES platforms(id),
  name        TEXT NOT NULL,    -- 'YouTube Shorts'
  pattern     TEXT NOT NULL,   -- '*youtube.com/shorts*'
  enabled     INTEGER NOT NULL DEFAULT 1
);

-- WireGuard devices
CREATE TABLE devices (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  name       TEXT NOT NULL,
  public_key TEXT NOT NULL UNIQUE,
  vpn_ip     TEXT NOT NULL UNIQUE,
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  last_seen  TEXT
);

-- ★ Blocking sessions (locked once started)
CREATE TABLE sessions (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  started_at   TEXT NOT NULL DEFAULT (datetime('now')),
  ends_at      TEXT,                       -- NULL = manual stop only
  state        TEXT NOT NULL DEFAULT 'active',
                                           -- 'active'|'unlock_pending'|'unlocked'|'emergency'|'ended'
  locked       INTEGER NOT NULL DEFAULT 1, -- 1 = no edits allowed
  created_at   TEXT NOT NULL DEFAULT (datetime('now'))
);

-- ★ Unlock attempts log (for escalation)
CREATE TABLE unlock_attempts (
```

```
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id   INTEGER REFERENCES sessions(id),
  platform_id  INTEGER REFERENCES platforms(id), -- NULL = global
  requested_at TEXT NOT NULL DEFAULT (datetime('now')),
  wait_seconds INTEGER NOT NULL,            -- how long they had to wait
  phrase_typed TEXT NOT NULL,              -- the friction phrase typed
  confirmed_at TEXT,                       -- NULL = abandoned
  expires_at   TEXT,                       -- auto-relock time
  relocked_at  TEXT
);

-- ★ Emergency unlock log
CREATE TABLE emergency_log (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id   INTEGER REFERENCES sessions(id),
  reason       TEXT NOT NULL,
  used_at      TEXT NOT NULL DEFAULT (datetime('now')),
  expired_at   TEXT NOT NULL,              -- emergency expires after 30 min
  week_number  TEXT NOT NULL               -- 'YYYY-Www' for weekly cap
);

-- ★ Schedule rules (locked while session active)
CREATE TABLE schedules (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  name       TEXT NOT NULL,
  enabled    INTEGER NOT NULL DEFAULT 1,
  days_mask  INTEGER NOT NULL,             -- bitmask: bit0=Sun..bit6=Sat
  start_time TEXT NOT NULL,               -- 'HH:MM'
  end_time   TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

-- Block statistics
CREATE TABLE stats_daily (
  date   TEXT NOT NULL,
  domain TEXT NOT NULL,
  count  INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY (date, domain)
);

CREATE TABLE stats_totals (
  domain       TEXT PRIMARY KEY,
  count        INTEGER NOT NULL DEFAULT 0,
  last_blocked TEXT NOT NULL DEFAULT (datetime('now'))
);

-- ★ Health event log
CREATE TABLE health_log (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  status     TEXT NOT NULL,   -- 'active'|'degraded'|'broken'
  detail     TEXT,
  checked_at TEXT NOT NULL DEFAULT (datetime('now'))
);
```

Non-Negotiable Feature Modules

★ Session Lock State Machine (`server/src/session-manager.ts`)

```
IDLE
  | start session
  ▼
ACTIVE (locked) ──────────────────────────────┐
  | request unlock (single platform, timed)   | auto-relock on expiry
```

```
  ▼                                          │
UNLOCK_PENDING (wait room, typed friction)    │
  │ wait elapsed + phrase typed               │
  ▼                                          │
UNLOCKED (platform X only, 15 min max) ──────┘
  │ emergency request (separate path)
  ▼
EMERGENCY (essential apps only, 30 min cap, logged)

Rules enforced server-side:
- No schedule edits while state != IDLE
- No platform toggle while state = ACTIVE (only via unlock flow)
- No global "disable all" button — only per-platform, timed unlock
- State survives server restart (stored in DB)
- Session re-activates on server restart if ends_at is in the future
```

★ Typed Friction (`server/src/friction.ts`)

Escalating difficulty based on unlock attempts today:

Attempt # today	Wait time	Phrase to type
1st	10 min	`"I am choosing to unlock [Platform]"`
2nd	20 min	`"I am breaking my commitment. This is attempt 2 today."`
3rd	40 min	`"I have no self-control right now. I am unlocking [Platform] for the third time today."`
4th+	60 min	Random 50-character string (no meaning, pure friction)

- Phrase must match exactly (case-sensitive, trimmed)
- Wrong phrase resets the input — wait timer continues
- Per-platform only — unlocking Instagram does NOT affect YouTube
- Auto-relock: unlocked state expires in 15 minutes, then re-locks automatically
- All attempts logged in `unlock_attempts` table (visible in dashboard shame log)

★ Emergency System (`server/src/emergency.ts`)

- **Weekly cap**: 3 emergency unlocks per calendar week (stored in `emergency_log`)
- **Essential apps only**: Emergency only whitelists a hardcoded set of domains:
- Phone/calls: (VoIP apps — Google Meet, Zoom, FaceTime domains)
- Maps: maps.google.com, apple.com/maps, waze.com
- Banking: user-configured list (up to 5 domains)
- Family communication: user-configured (up to 3 contacts' app domains)
- **NOT unlocked by emergency**: Any platform in the block list
- **Duration**: 30 minutes, then auto-relock
- **Logged prominently**: Emergency log shown on dashboard with shame indicator
- **Does not affect schedules**: Future scheduled blocks are not cancelled

★ Time-Change Protection (`server/src/time-guard.ts`)

- On startup and every 15 minutes: compare `Date.now()` against NTP response from `pool.ntp.org`
- If system clock is more than 5 minutes behind NTP → flag as tamper attempt
- Action: log tamper event, re-engage blocking immediately, alert in dashboard
- All schedule evaluations use NTP-verified time, not `Date.now()` alone

- Schedules stored and evaluated in UTC to prevent timezone-manipulation bypass

★ Reliability Monitor (`server/src/health-monitor.ts`)

Every 5 minutes cron job checks: 1. DNS server is listening on WireGuard interface (`10.13.13.1:53`) 2. WireGuard interface `wg0` is up (`wg show wg0` exit code = 0) 3. Block set in memory is non-empty 4. Database is readable/writable 5. NTP reachable (within 30s)

Status → `settings.health_status` : - `active` : All checks pass - `degraded` : 1-2 checks failed (DNS or WireGuard flapping) - `broken` : Core blocking non-functional

Dashboard shows prominent colored status badge. Push notification if status degrades (webhook-based, configurable URL).

★ Restart-Safe Persistence

- All state in SQLite (no in-memory-only state for blocking decisions)
- On startup: restore session state from DB, re-engage blocking if session was active
- `blockme.service` systemd unit with `Restart=always` , `RestartSec=3`
- WireGuard `wg-quick@wg0.service` enabled at boot
- WireGuard client config sets `PersistentKeepalive=25` for auto-reconnect
- iOS/Android: configure WireGuard with "On Demand" enabled → reconnects after any network change

★ Circumvention Prevention

Attack vector	Mitigation
DNS-over-HTTPS (DoH)	Add `dns.google` , `cloudflare-dns.com` , `dns.quad9.net` to blocklist; block port 853 at router
IPv6 bypass	Both A and AAAA returned as `0.0.0.0` / `::`
Subdomain bypass	Suffix-walk algorithm blocks all subdomains automatically
VPN disconnect	Health monitor detects, alerts; "On Demand" reconnects automatically
Incognito/private	VPN-level — invisible to browsers; affects all browsing modes
Embedded iframes	Domain blocked at DNS level regardless of embed context
App-level bypass	Full-tunnel VPN (`AllowedIPs = 0.0.0.0/0`) — no split tunnel
Clock manipulation	NTP comparison every 15 min; blocks re-engage on detected tamper
Editing WireGuard config	Peer removal via dashboard also removes from live `wg` config

---

Platforms (Pre-Seeded)

Platform	Category	Key Domains
Twitter/X	social	twitter.com, x.com, t.co, twimg.com
Instagram	social	instagram.com, cdninstagram.com
Facebook	social	facebook.com, fb.com, fbcdn.net, messenger.com
TikTok	video	tiktok.com, tiktokcdn.com, musical.ly
YouTube	video	youtube.com, youtu.be, ytimg.com, googlevideo.com
Reddit	social	reddit.com, redd.it, redditstatic.com
Snapchat	messaging	snapchat.com, sc-cdn.net

Platform	Category	Key Domains
Discord	messaging	discord.com, discordapp.com, discord.gg
LinkedIn	professional	linkedin.com, licdn.com
Pinterest	social	pinterest.com, pinimg.com
Twitch	video	twitch.tv, jtvnw.net
Threads	social	threads.net
WhatsApp	messaging	whatsapp.com, whatsapp.net
Telegram	messaging	telegram.org, t.me
Bluesky	social	bsky.app, bsky.social
BeReal	social	bereal.com

Sub-platform URL rules (browser extension): - YouTube Shorts → `*youtube.com/shorts*` - YouTube Reels → `*youtube.com/reels*` - Reddit Live → `*reddit.com/live*` - Twitter Trending → `*twitter.com/i/trending*`

API Endpoints

Method	Path	Auth	Lock-aware	Description
POST	`/api/auth/login`	No	No	Get JWT (rate: 5/15min)
GET	`/api/auth/status`	No	No	Token validity
GET	`/api/settings`	No	No	Settings + health state
PUT	`/api/settings`	Yes	Yes	Update (blocked during locked session)
PUT	`/api/settings/password`	Yes	No	Change password
GET	`/api/session`	No	No	Current session state + cooldown
POST	`/api/session/start`	Yes	No	Start a blocking session
POST	`/api/session/unlock-request`	Yes	Yes	Begin unlock flow (per-platform)
POST	`/api/session/unlock-confirm`	Yes	Yes	Submit typed phrase, get timed unlock
POST	`/api/session/emergency`	Yes	Yes	Emergency unlock (essential only, capped)
GET	`/api/session/unlock-log`	Yes	No	Shame log of all unlock attempts
GET	`/api/session/emergency-log`	Yes	No	Emergency usage history
GET	`/api/devices`	Yes	No	List VPN devices
POST	`/api/devices`	Yes	No	Add device + WG config + QR
GET	`/api/devices/:id/config`	Yes	No	Download .conf
GET	`/api/devices/:id/qr`	Yes	No	QR code PNG
DELETE	`/api/devices/:id`	Yes	No	Remove device
GET	`/api/blocklist`	No	No	All platforms + domains + url_rules
POST	`/api/blocklist/domains`	Yes	Yes	Add custom domain
PATCH	`/api/blocklist/domains/:id`	Yes	Yes	Toggle (blocked in session)
DELETE	`/api/blocklist/domains/:id`	Yes	Yes	Remove (blocked in session)

Method	Path	Auth	Lock-aware	Description
PATCH	`/api/blocklist/platforms/:id`	Yes	Yes	Toggle platform (blocked in session)
PATCH	`/api/blocklist/url-rules/:id`	Yes	Yes	Toggle URL rule
GET	`/api/schedules`	No	No	List schedules
POST	`/api/schedules`	Yes	Yes	Create (blocked during active session)
PUT	`/api/schedules/:id`	Yes	Yes	Update (blocked during active session)
DELETE	`/api/schedules/:id`	Yes	Yes	Delete (blocked during active session)
GET	`/api/stats/summary`	No	No	Today/week/alltime
GET	`/api/stats/daily`	No	No	Per-day counts
GET	`/api/stats/domains`	No	No	Top blocked domains
DELETE	`/api/stats`	Yes	No	Clear stats
GET	`/api/health`	No	No	Health status (active/degraded/broken)

Frontend Pages

Dashboard

- Health badge (green/yellow/red): "Protected" / "Degraded" / "Broken"
- Session status: "Blocking Active — Session started 2h ago"
- Stats cards: Blocked Today / This Week / All Time
- Bar chart (Recharts): last 7 days
- Top blocked domains
- Unlock shame log: "You tried to unlock Instagram 3 times today"
- Emergency remaining: "2 of 3 emergency unlocks remaining this week"

Unlock Page (★ key UX)

- Step 1: Select platform to unlock (not global)
- Step 2: "Wait room" — countdown timer (escalates each attempt)
- Step 3: Typed friction phrase displayed → user must type it exactly
- Step 4: 15-minute timed unlock begins, auto-relock countdown visible
- All steps tracked, no back button

Emergency Page

- Shows remaining emergency count (e.g. "2 of 3 left this week")
- Requires typing "EMERGENCY" to confirm
- Only shows essential app domains, not social platforms
- 30-minute timer, auto-relock

Devices Page

- Add device → QR code → scan with WireGuard app
- Setup instructions per OS (iOS: On Demand, Android, Windows, Mac)
- Device list with last seen timestamp

Blocklist Page

- Platform cards grouped by category
- Each platform: enabled/disabled toggle (greyed out + locked icon during session)
- Expand to see domains + URL rules (YouTube Shorts toggle, etc.)
- Custom domain input

Schedules Page

- Schedule cards: day pills + time range
- Locked indicator during active session ("Cannot edit — session active")
- New schedule modal (day checkboxes + time pickers)

Settings Page

- Emergency cap setting (3/5/10 per week)
- Weekly emergency count reset day
- Essential domains configuration
- NTP server setting
- Change password
- Server IP (used in WG configs)
- Health webhook URL
- Clear stats

Implementation Sequence

1. Root `package.json` (workspaces)
2. `server/package.json` + `tsconfig.json`
3. `server/src/db/schema.ts` — all tables + seed data
4. `server/src/db/index.ts` — SQLite singleton + WAL
5. `server/src/db/queries.ts` — all typed query functions
6. `server/src/blocklist.ts` — domain list + `isDomainBlocked()`
7. `server/src/time-guard.ts` — NTP check + tamper detection
8. `server/src/scheduler.ts` — NTP-verified schedule checker
9. `server/src/dns-server.ts` — DNS on WireGuard interface
10. `server/src/wireguard.ts` — keypair, peer add/remove, config gen
11. `server/src/session-manager.ts` — session lock state machine
12. `server/src/friction.ts` — typed friction phrase generator + escalation
13. `server/src/emergency.ts` — emergency unlock system
14. `server/src/health-monitor.ts` — 5-min health check cron
15. `server/src/routes/auth.ts` — login + JWT + rate limit
16. `server/src/routes/session.ts` — unlock flow + emergency
17. `server/src/routes/devices.ts` — VPN peer + QR
18. `server/src/routes/blocklist.ts` — platform/domain toggles (lock-aware)
19. `server/src/routes/schedules.ts` — schedule CRUD (lock-aware)
20. `server/src/routes/stats.ts` + `health.ts`
21. `server/src/index.ts` — wire everything
22. `client/` — Vite + Tailwind setup

23. `client/src/` — AuthContext + api/client.ts + routing
24. `client/src/pages/` — all 8 pages
25. `extension/` — manifest + background + popup
26. `blockme.service` — systemd unit
27. `README.md` — setup + WireGuard install + router config

Critical Files

- `server/src/session-manager.ts` — session lock state machine (no bypasses)
- `server/src/friction.ts` — escalating typed friction
- `server/src/emergency.ts` — capped emergency system
- `server/src/time-guard.ts` — clock tamper detection
- `server/src/health-monitor.ts` — reliability monitoring
- `server/src/dns-server.ts` — core DNS blocking engine
- `server/src/wireguard.ts` — VPN peer management
- `server/src/db/schema.ts` — complete data model
- `server/src/routes/session.ts` — unlock flow API
- `client/src/pages/UnlockPage.tsx` — typed friction UX
- `client/src/pages/EmergencyPage.tsx` — emergency UX

Verification

1. `sudo npm run dev` — start server
2. `dig @10.13.13.1 instagram.com` — returns `0.0.0.0` ✓
3. `dig @10.13.13.1 google.com` — returns real IPs ✓
4. Add device → scan QR → connect WireGuard on phone
5. Open Instagram app on phone (VPN on, 4G) — fails to load ✓
6. Start session → try editing schedule → blocked ✓
7. Request unlock for Instagram → wait room appears → type phrase → 15-min unlock ✓
8. Try to unlock YouTube while Instagram is unlocked → still blocked ✓
9. Auto-relock after 15 min → Instagram blocked again ✓
10. Request emergency → essential apps only → logged ✓
11. Change system clock backward → tamper detected → blocks re-engage ✓
12. Kill server → restart → session still active ✓
13. `GET /api/health` → `{ status: "active" }` ✓